Trajectoires d’Avions en phase d’atterrissage

Gilles BAYALA ,11580

Option SII



Tour de contrôle

Extraite de larousse.fr



Asiana Airlines 214

Extraite de PSP Spotter, Youtube

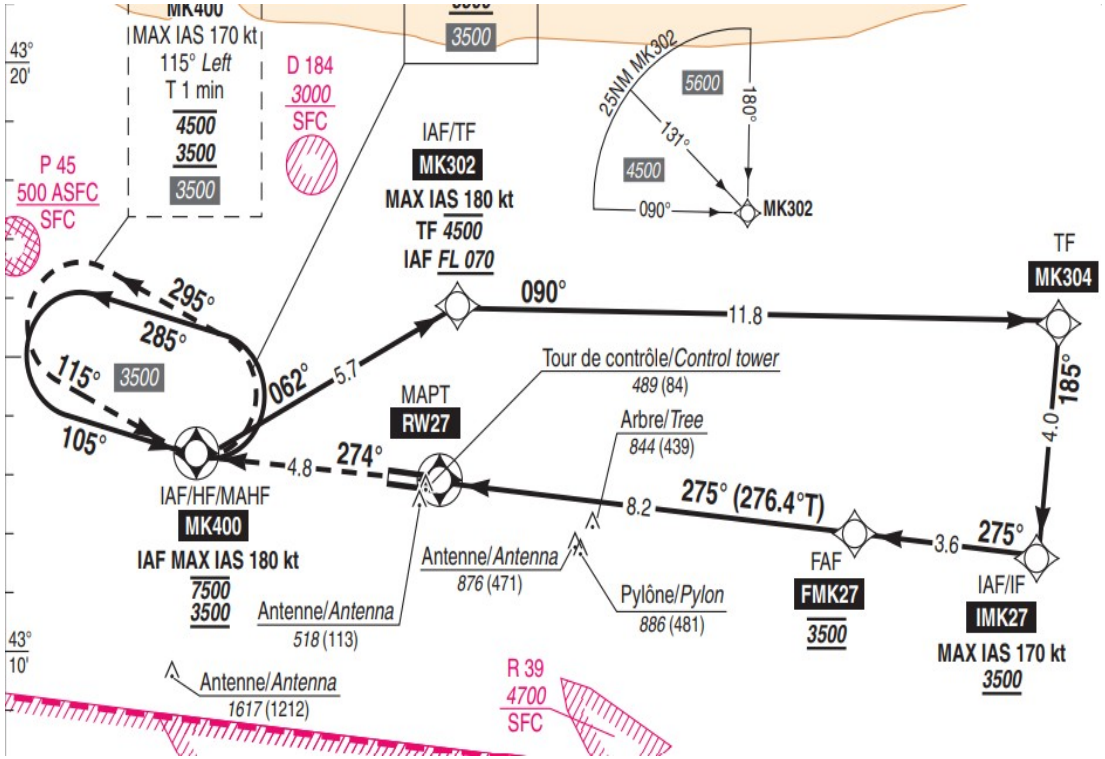
Problématique : Comment modéliser la trajectoire d’atterrissage d’un avion en respectant les contraintes physiques ?

Sommaire

I- Choix de l’approche
II- Optimisation et résultats
III- Conclusion

Choix de l’approche

Balises créées par la tour de contrôle



Aéroport de Carcassonne

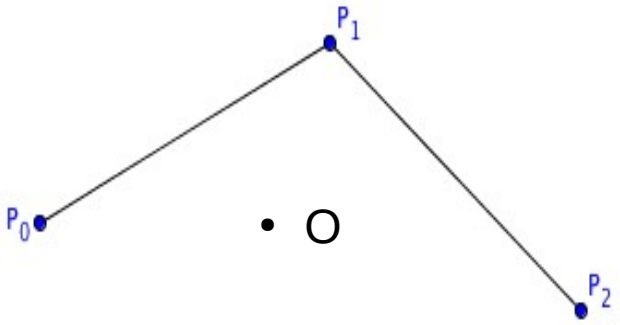
De J-P Fanguin, contrôleur aérien

Courbe de Bézier

Extraite de bibmath.com

$$\overrightarrow{OM_0}=(1-t)\overrightarrow{OP_0}+t\overrightarrow{OP_1}$$

$$\overrightarrow{OM_1}=(1-t)\overrightarrow{OP_1}+t\overrightarrow{OP_2}$$



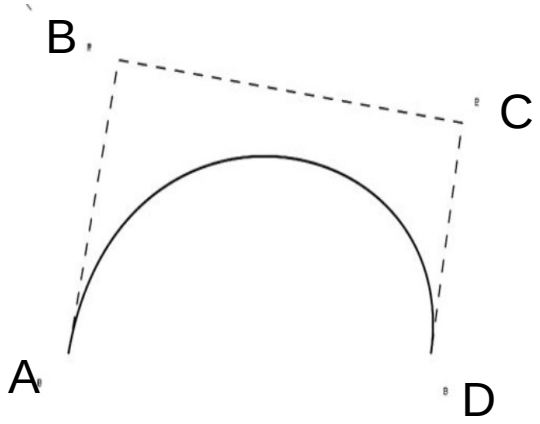
$$\overrightarrow{OM(t)}=(1-t)\overrightarrow{OM_0}+t\overrightarrow{OM_1}$$

$$\overrightarrow{OM(t)}=(1-t)^2\overrightarrow{OP_0}+2t(1-t)\overrightarrow{OP_1}+t^2\overrightarrow{OP_2}$$

Courbe de Bézier

$\forall n \in N/\{0\},\ \forall k \in [\![0;n]\!]\ \forall (A_i)_{i\in[\![0;n]\!]} \in \mathbb{R}^2$

$$B_k^n = \binom{n}{k} X^k (1-X)^{n-k}$$

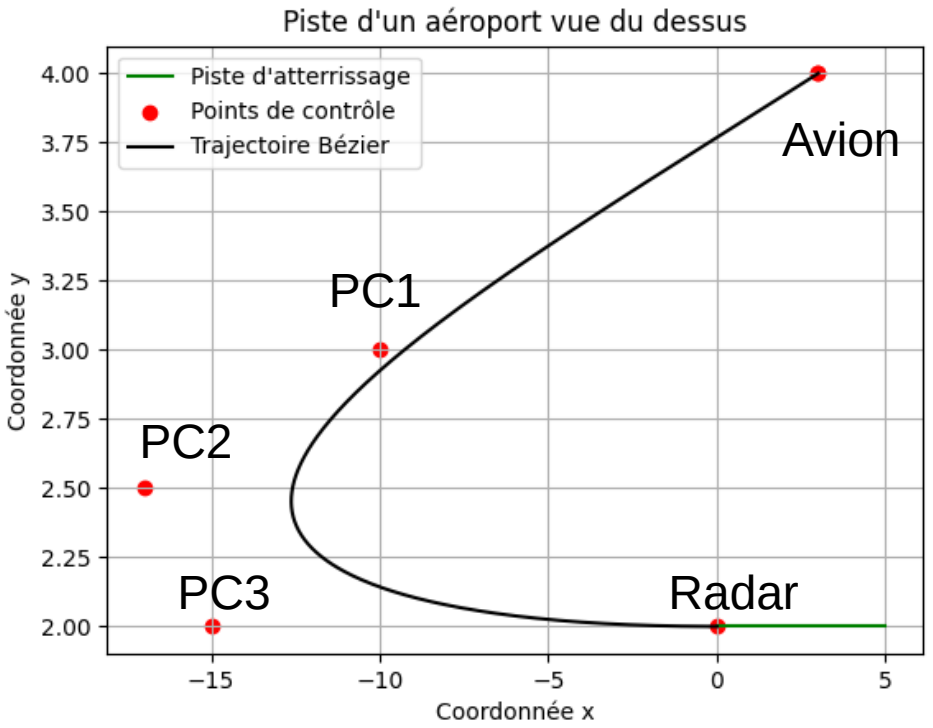


Extraite de techno-science.com

$$\boldsymbol{C} = \left\{ \sum_{k=0}^{n} B_k^n(t)\, A_k, t \in [0,1] \right\} \textit{ Courbe de Bézier associée à } (A_0, A_1, \ldots, A_n)$$

Problème : Comment placer les points de contrôle ?

Étude spécifique à 3 points de contrôle



Courbe de Bézier pour points de contrôle et avion fixes

Sommaire

I- Choix de l’approche
II- Optimisation et résultats
III- Conclusion

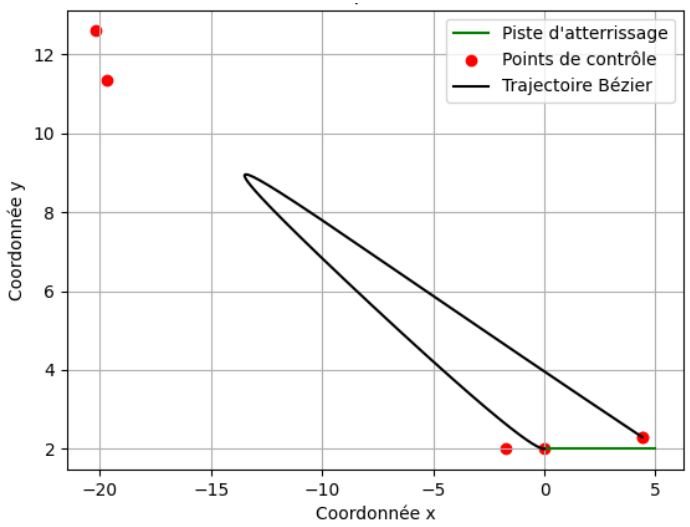


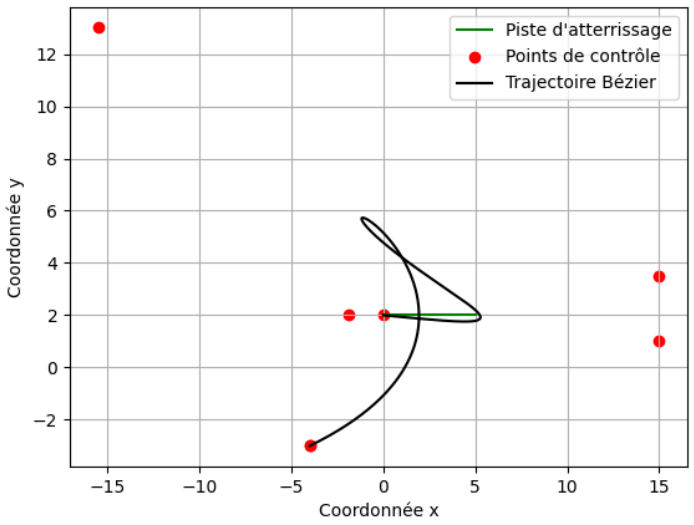
Fonction coût

3 paramètres :

- *Distance Avion-Radar*
- *Variation de courbure*
- *Courbure maximale*

Types de trajectoires prohibées





Calcul du coût

```
# Critères physiques
max_courbure = max(courbure(trajectoire))/100
rayon_min = 1 / max_courbure if max_courbure != 0 else float('inf')
distance = dist(trajectoire)/10
variation = variationcourbure(trajectoire)/10

# Exclusion des virages irréalistes (trop serrés)
if rayon_min < 4:  # rayon de courbure minimal imposé
    return 1e6  # coût énorme = élimination

# Pondération (à ajuster selon priorité)
d = 0.01  # poids de la courbure max
e = 1.0    # poids de la distance (peut être 0)
f = 1.0 # poids de la variation de courbure

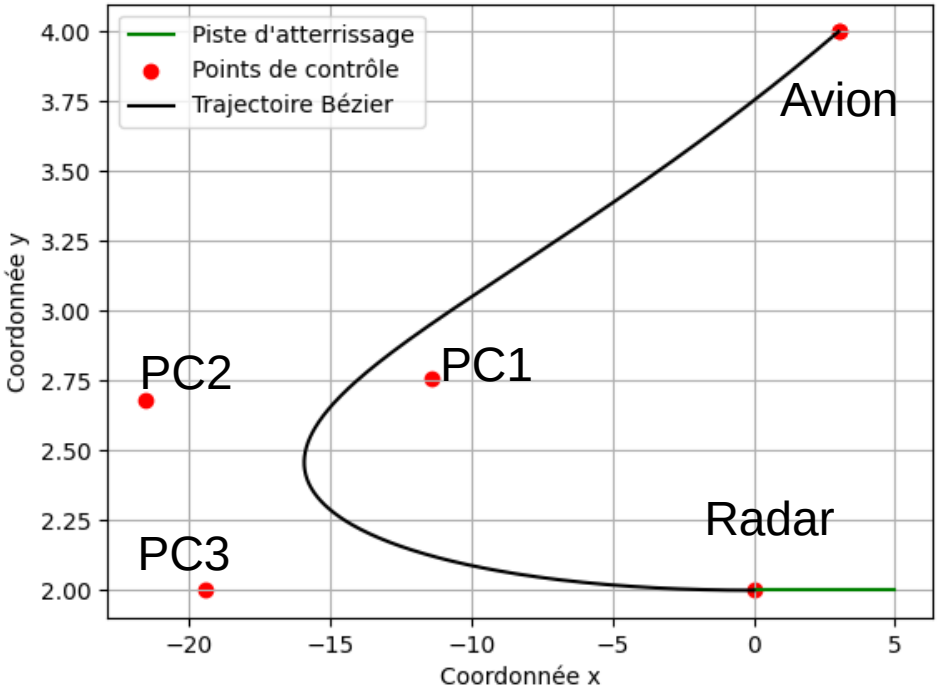
return d * max_courbure + e * distance + f * variation
```

Calcul selon la moyenne pondérée

Extraite des lignes 185 à 214

Génération aléatoire des Points de contrôle

Poids
Distance= 1.0
Variation Courbure= 1.0
Courbure Maximale= 0.01
Nombre d'itérations= 300

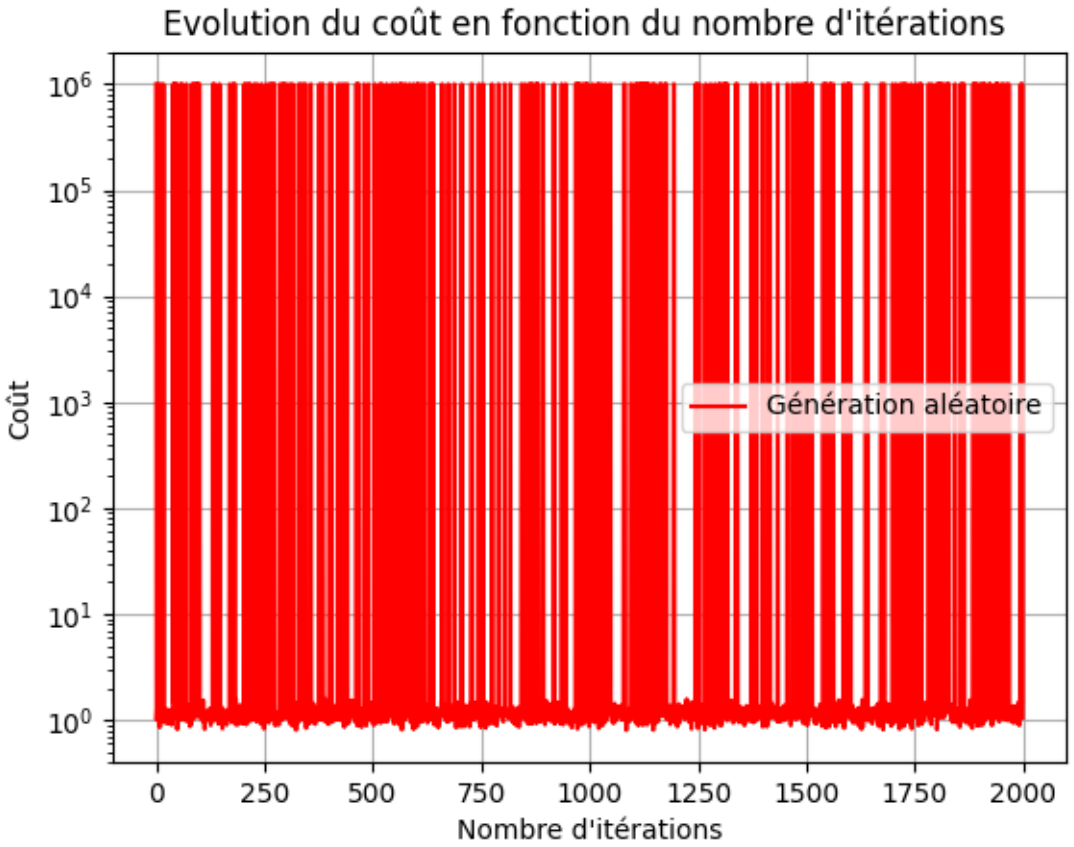


- Trajectoire réaliste
- Distance parcourue courte

Coût= 1.28
Distance=26.3 km

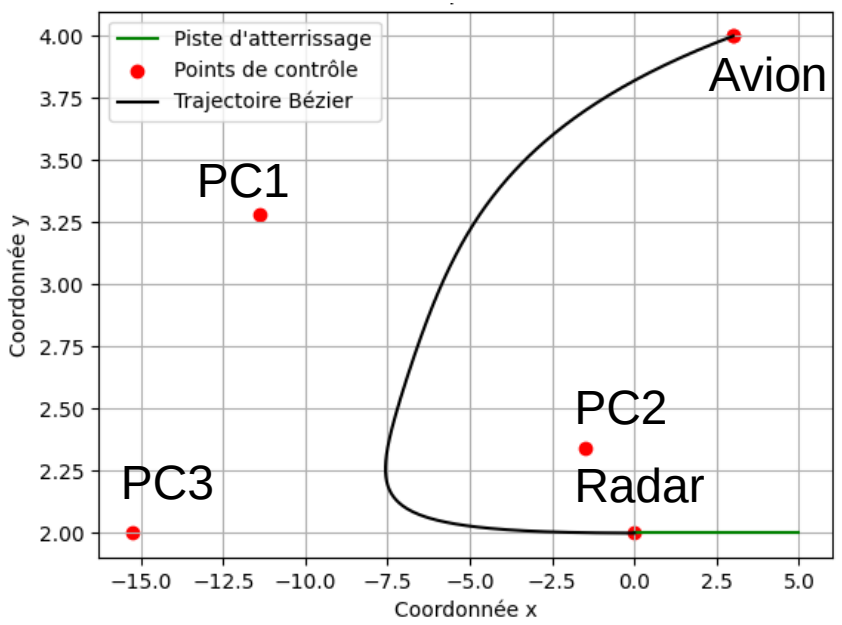
Génération aléatoire des points de contrôle

Recherche non optimale



Génération intelligente des Points de contrôle

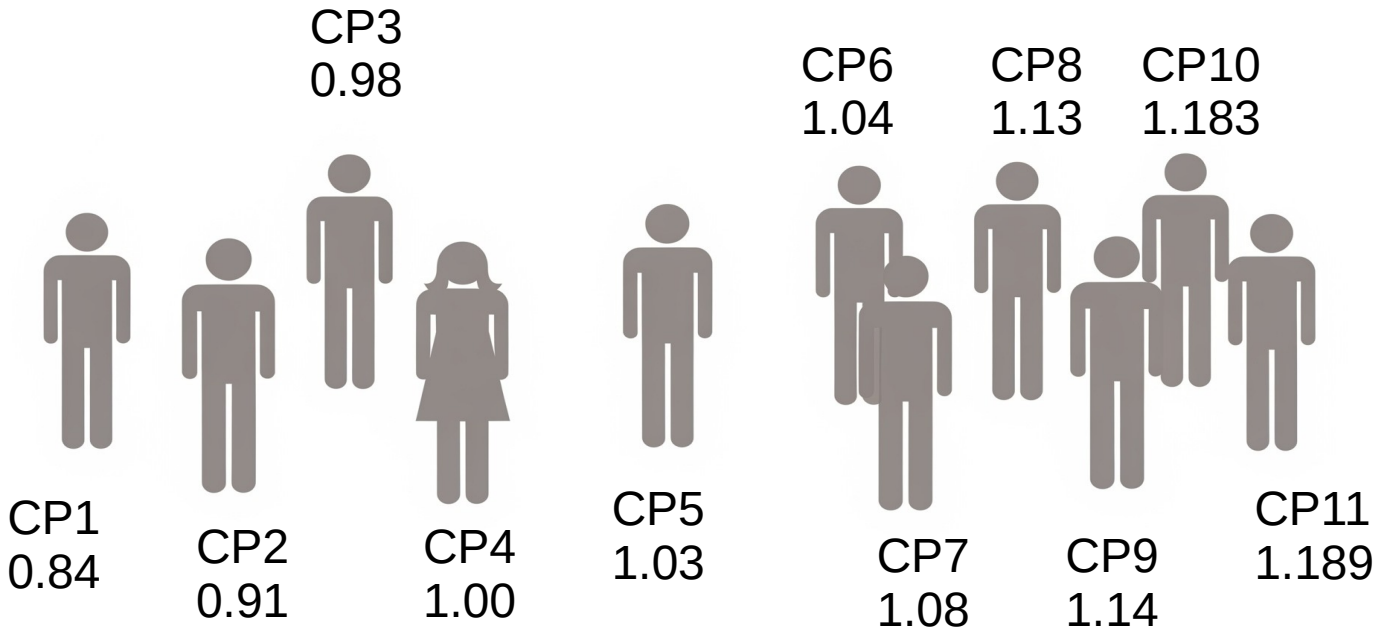
Poids
Distance= 1.0
Variation Courbure= 1.0
Courbure Maximale= 0.01
Nombre d’itérations= 300



- Trajectoire réaliste
- Distance parcourue plus longue
- Coût faible
- Recherche optimale

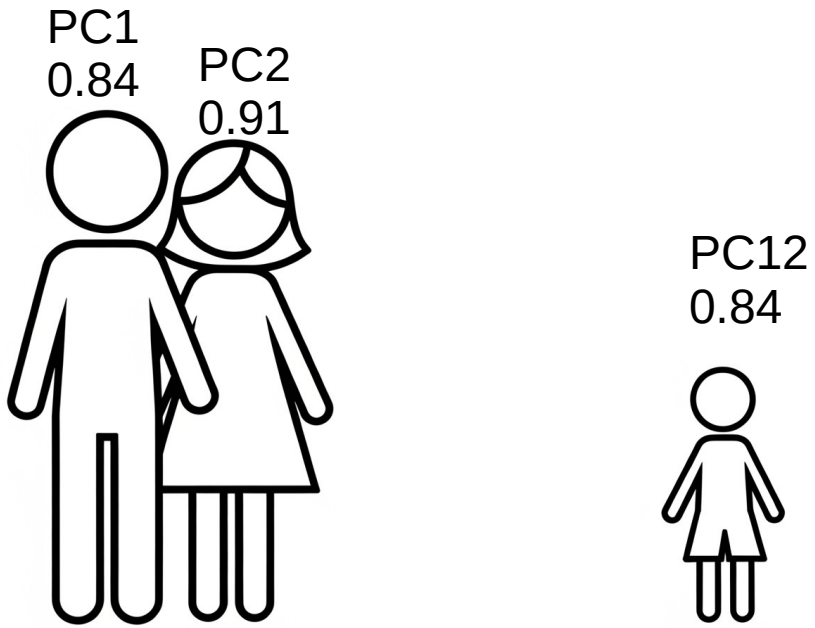
Coût= 0.80
Distance= 27.7 km

Algorithme Génétique



*C.P : Système de points de contrôle

Algorithme Génétique




```
def crossover(ind1, ind2) -> list:
    """Permet de mettre encommun les gènes de 2 trajectoires pour en obtenir
une nouvelle possédant un coût intéressant"""
    alpha=random.random()
    enfant=[(alpha*k1[0]+(1-alpha)*k2[0]
        ,alpha*k1[1]+(1-alpha)*k2[1])
        for k1,k2 in zip(ind1,ind2)
        ]
    return enfant
```

Algorithme Génétique

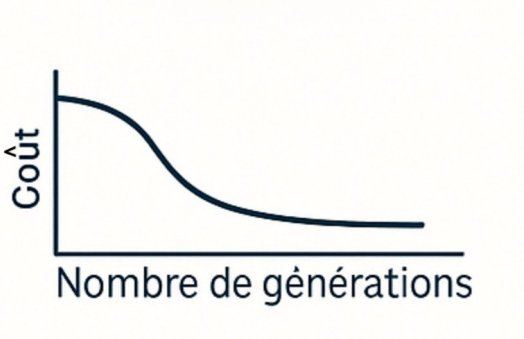
Sélection
(suivant la valeur du coût)

Croisements
(Parents → Enfants)

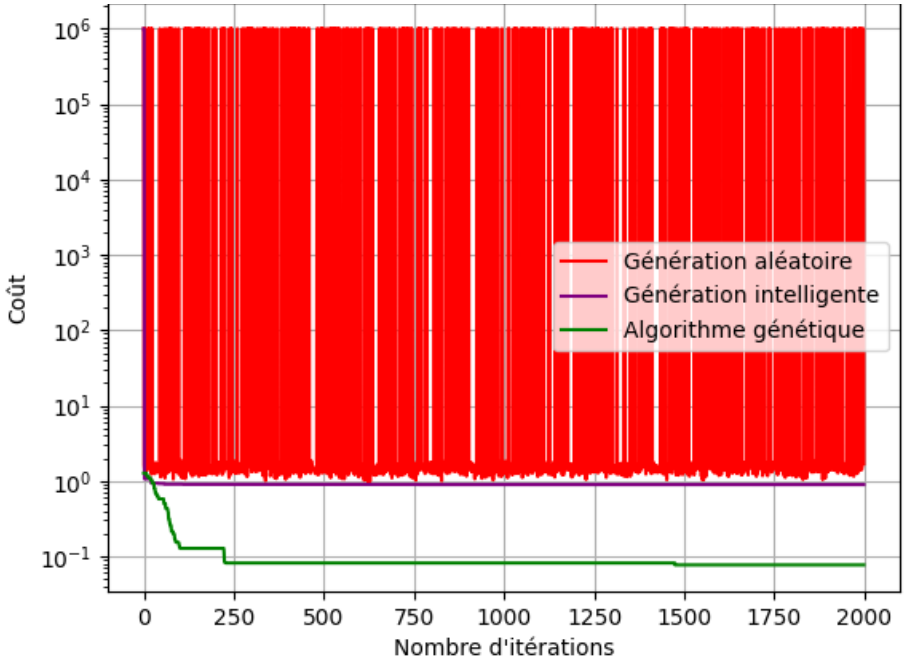
Nouvelle population
(issue des croisements)

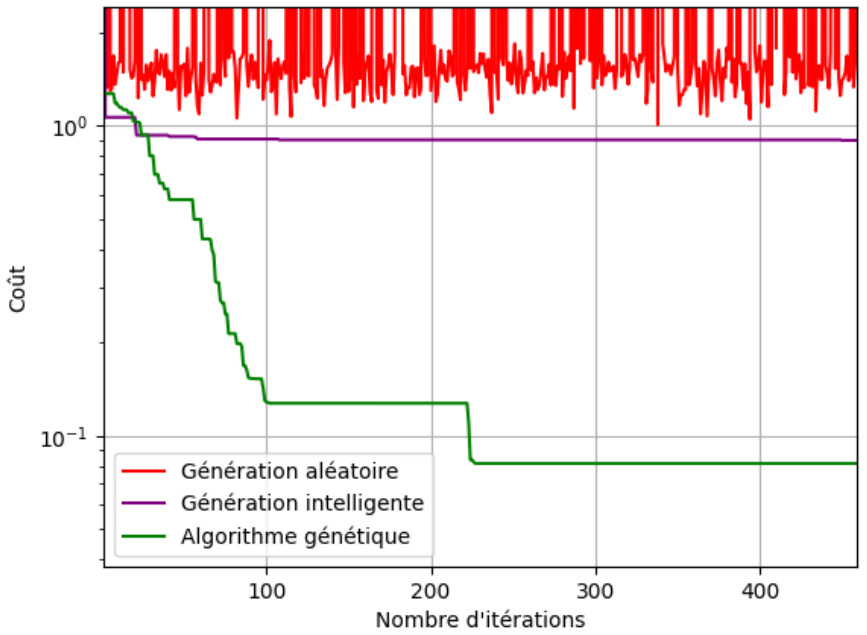
Mutation
(petite modification aléatoire des points)

Évaluation
(calcul du coût)

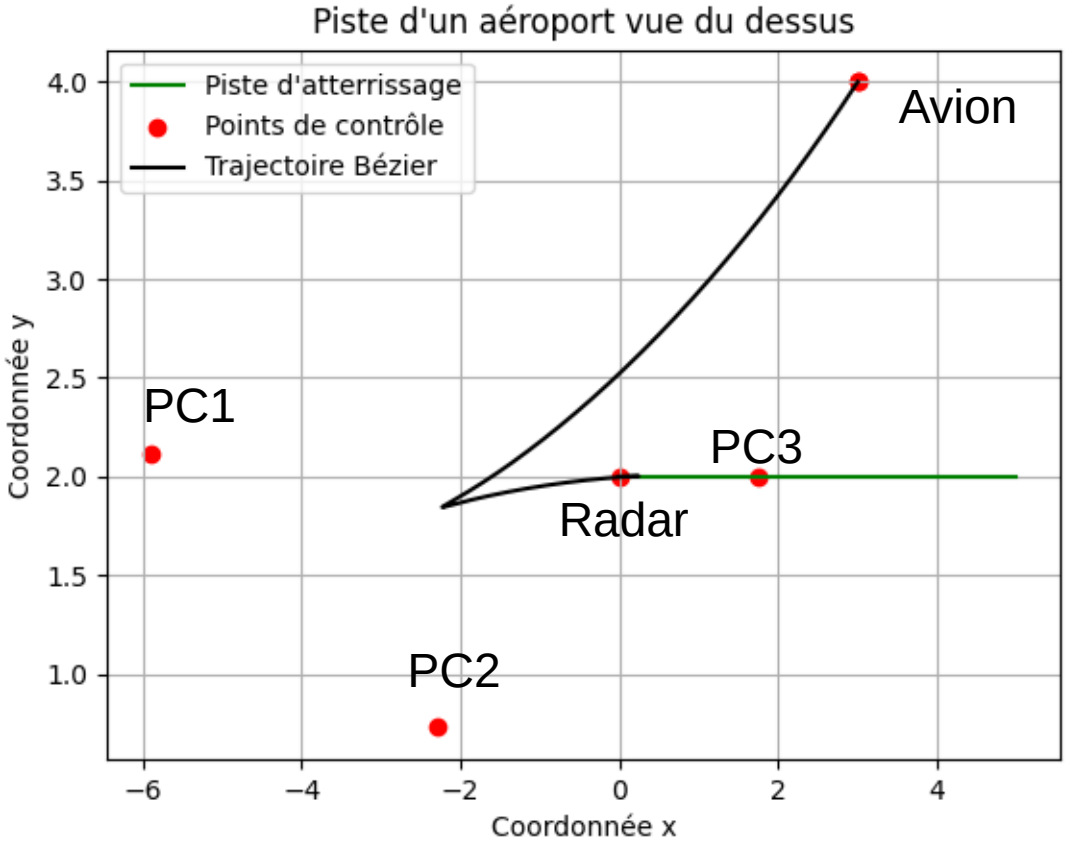


Comparaison des approches d’optimisation des trajectoires d’atterrissage





Trajectoire d’avion via A.G après 300 générations

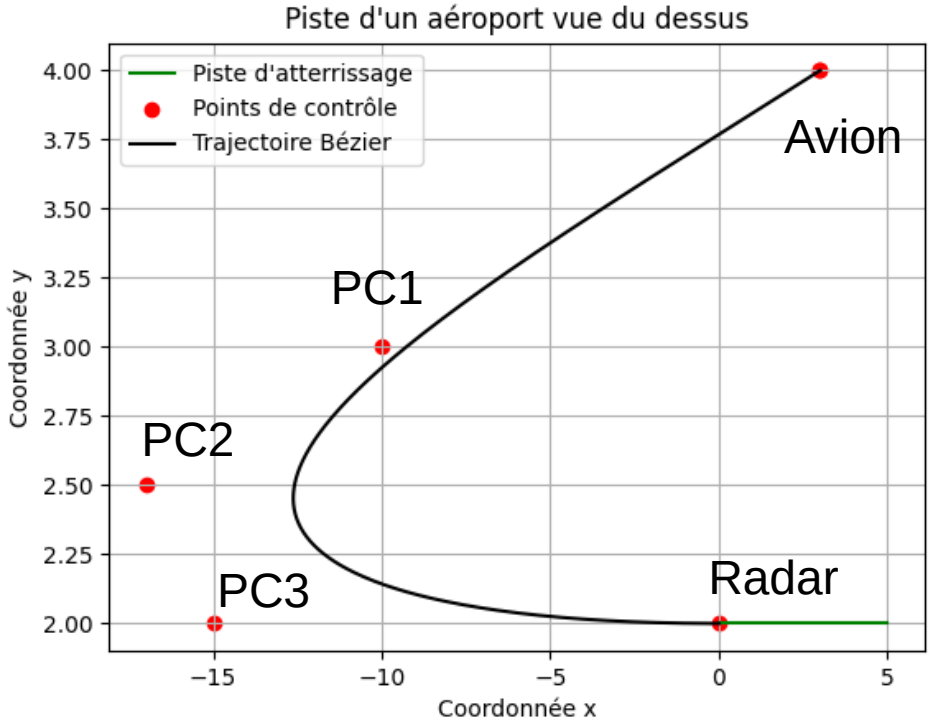


Coût= 0.34
Distance= 10.4 km

- Coût minime
- Distance parcourue faible
- Trajectoire irréaliste !

Alternative 1 : Contraintes supplémentaires

Assurer une chronologie des points de contrôle

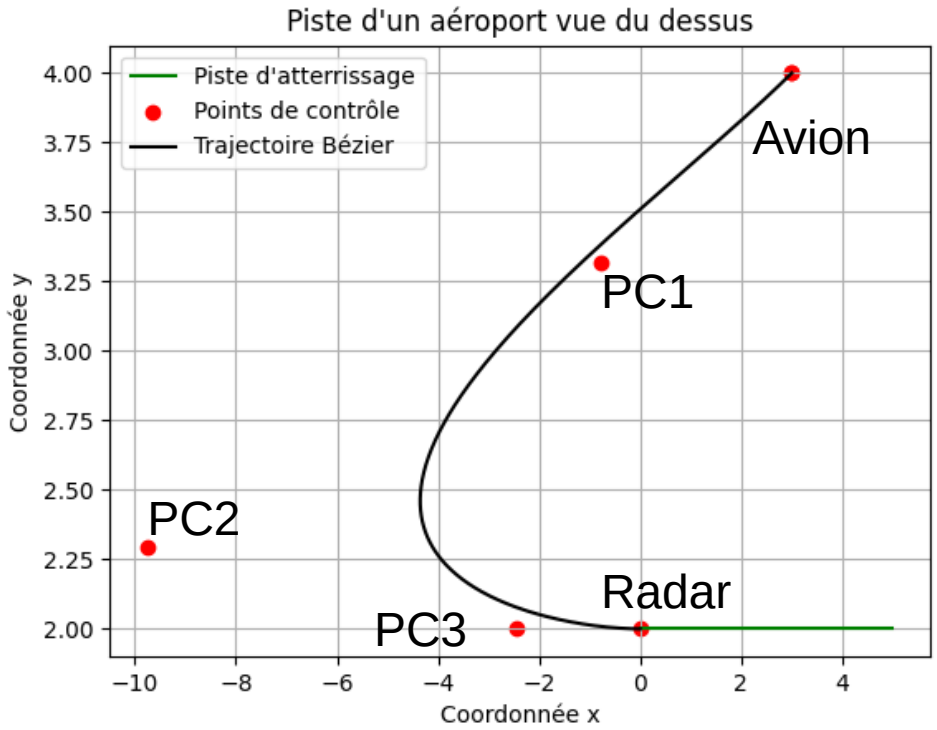


Exemple de points fixés manuellement

Alternative 1 : Contraintes supplémentaires

Poids
Distance= 1.0
Variation Courbure= 1.0
Courbure Maximale= 0.01

Distance = 12.2 km
Coût = 0.45

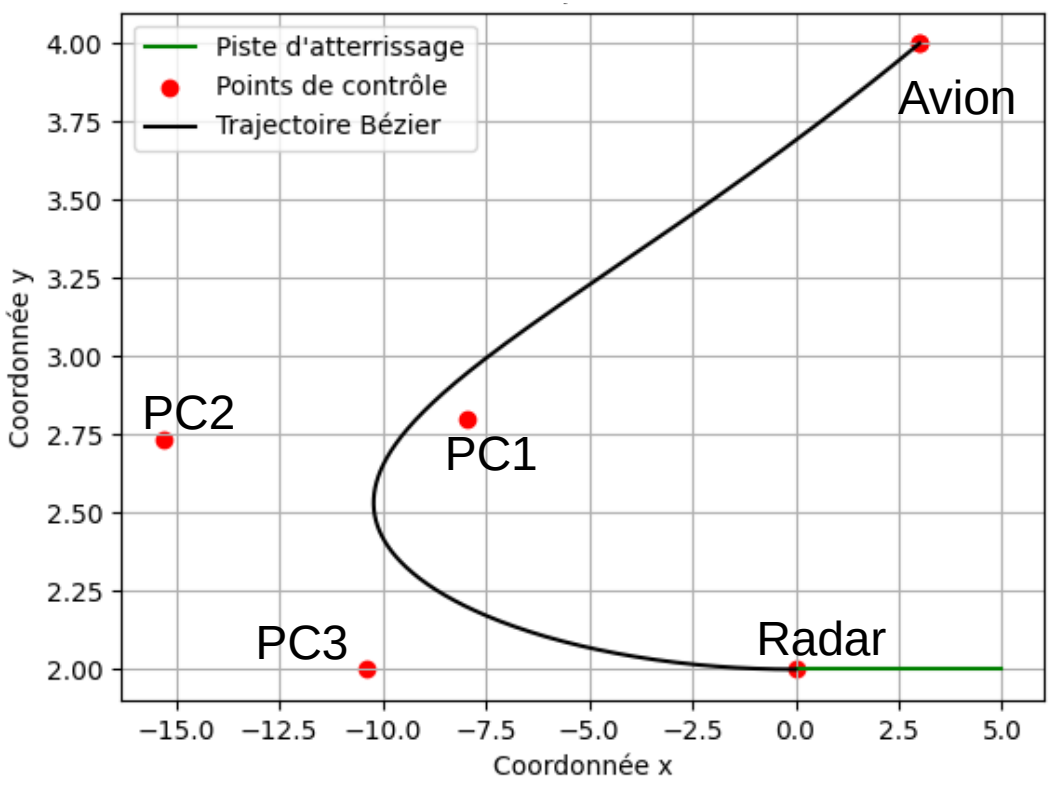


Trajectoire obtenue par Algorithme Génétique pour 300 générations

Alternative 2 : Baisser le nombre de générations

Poids
Distance= 1.0
Variation Courbure= 1.0
Courbure Maximale= 0.01

Coût= 0.80
Distance= 17.7 km



Trajectoire issue d'un A.G
après 3 générations

Sommaire

I- Choix de l’approche
II- Optimisation et résultats
III- Conclusion



Conclusion

- Modélisation par courbe de Bézier intéressante
- Fonction coût : rôle clé
- 3 dimensions: Complexifie le problème

Références images

- Diapositive 1: Tour de contrôle
- Diapositive 1: Asiana 214
- Diapositive 5: Courbe de Bézier
- Diapositive 6: Courbe de Bézier
- Diapositive 28: Atterrisage 3D

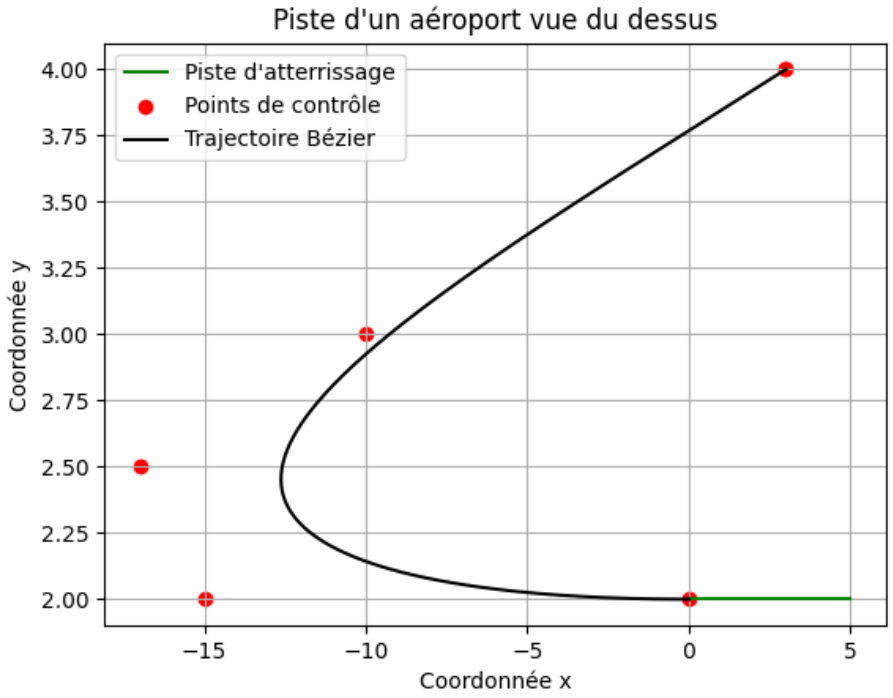
Références TIPE

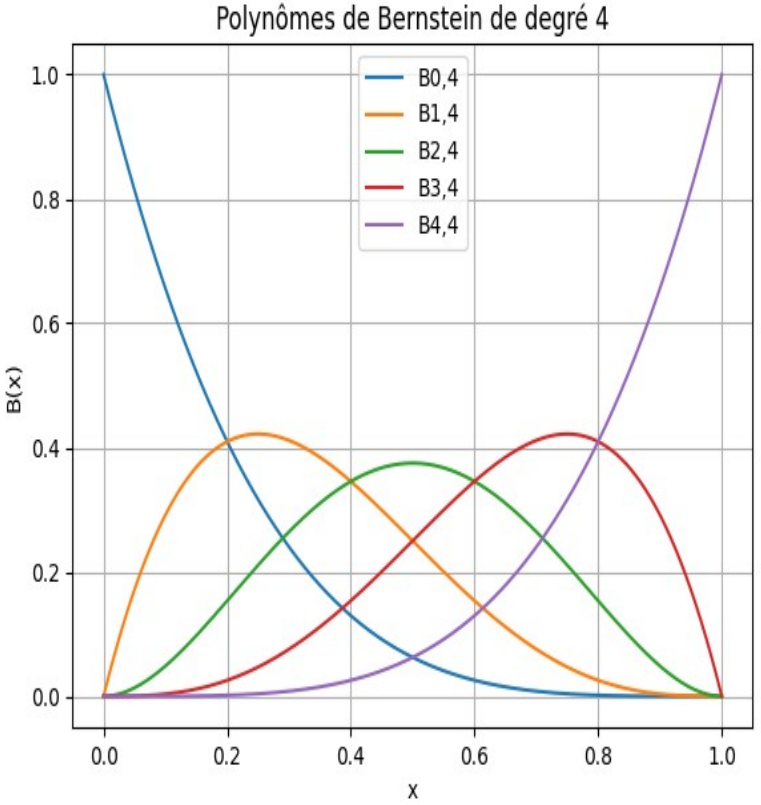
- [1] GIRARDET : Trafic aérien : détermination optimale et globale des trajectoires d'avion en présence de vent. : Thèse de doctorat, Université de Toulouse, Institut National des Sciences Appliquées, 2014.
- [2] IVAO FRANCE :Circuit de piste; consulté le 19/10/24
- [3] FARIN, HOSCHECK : **Handbook of Computer Aided Geometric Design** : ISBN: 978-0444511040 (2002)
- [4] LORENZI : TP sur les courbes de Bézier
- [5] KELAM : Rayon de courbure
- [6] ALLIOT , DURAND : Algorithmes génétiques : 14 mars 2005

Annexes

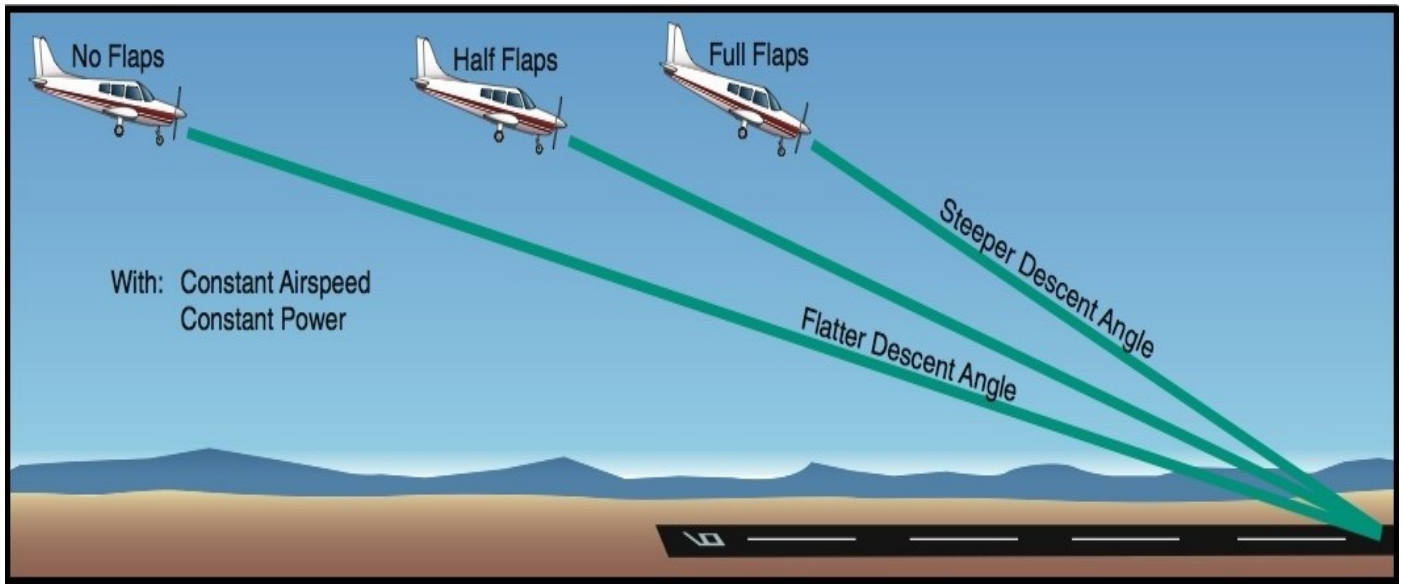
1) Polynômes de Bernstein
2) Approches d’avion en 3D
3) Rayon minimal
4) Position aléatoire de l’avion
5) Script Python

Annexe 1 : Polynômes de Bernstein





Annexe 2 : Approches d’avion en 3D



Extraite de piloter.ca

Annexe 3 : Rayon minimal

```
194     # Exclusion des virages irréalistes (trop serrés)
195     if rayon_min < 4:  # rayon de courbure minimal imposé
196         return 1e6  # coût énorme = élimination
197
```

Dû à l'accélération centripète $r = \frac{v^2}{a_c * \tan(\Phi)}$

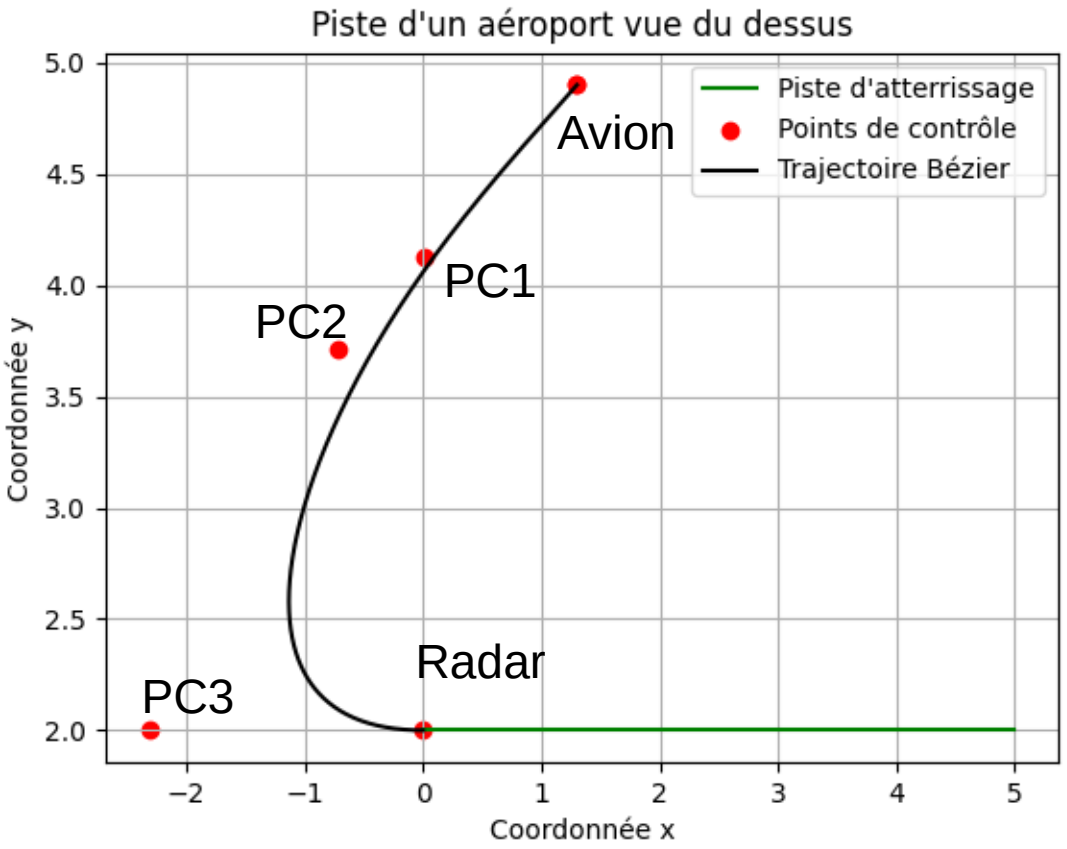
Extrait de lavionnaire.fr

v= Vitesse
r= Rayon de courbure
Φ= Angle d'inclinaison de l'avion

Annexe 4 : Position aléatoire de l'avion

Poids
Distance= 1.0
Variation Courbure= 1.0
Courbure Maximale= 0.01
Nombre d'itérations= 300

Distance= 3.14 km
Coût= 0.19



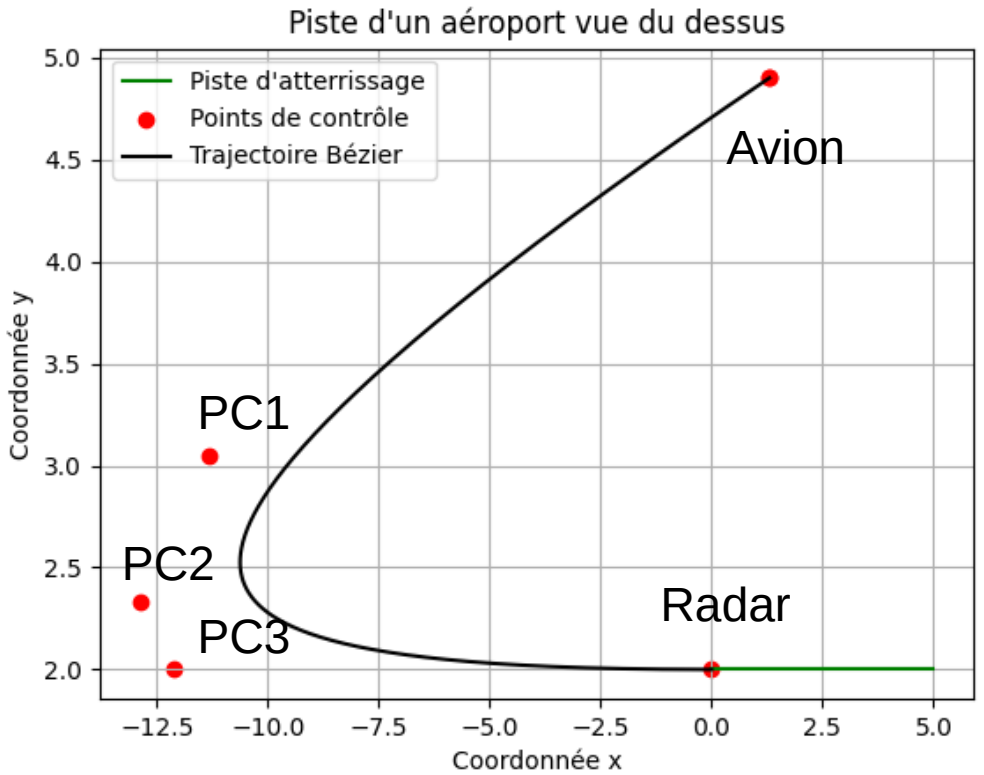
Génération des Points de contrôle par algorithme génétique

Annexe 4 : Position aléatoire de l’avion

Poids
Distance= 1.0
Variation Courbure= 1.0
Courbure Maximale= 0.01
Nombre d’itérations= 300

Distance= 17.6 km
Coût= 0.99

Génération aléatoire des Points de contrôle

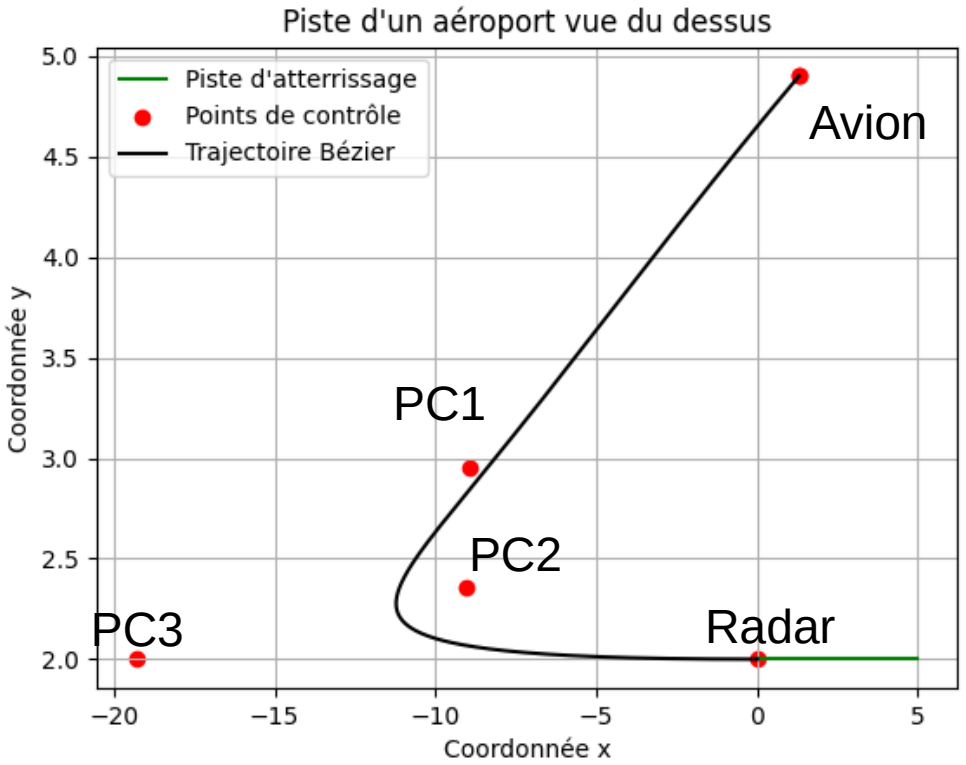


Annexe 4 : Position aléatoire de l’avion

Poids
Distance= 1.5
Variation Courbure= 0.5
Courbure Maximale= 0.01
Nombre d’itérations= 3

Distance= 24.02 km
Coût= 0.96

Génération ‘intelligente’ des Points de contrôle



Annexe 5 : Script Python

```python
# TIPE 11580 VF.py

import math as m
import matplotlib.pyplot as plt
import numpy as np
import random


# Coordonnées radar (début de piste)
radar_x, radar_y = 0, 2
Radar = (radar_x, radar_y)


# Position aléatoire de l'avion
C = np.arange(0, 5, 0.1)
D = np.arange(2,5,0.1)
avion_x = random.choice(C)
avion_y = random.choice(D)
Avion = (avion_x, avion_y)


# # #Position fixe de l'avion
# avion_x=3
# avion_y=4
# Avion = (avion_x, avion_y)


# Piste
X = np.linspace(0, 5, 500)
piste_y = [2] * 500


##Fonctions auxiliaires pour les polynômes de Bernstein et Bézier


# Fonction binomiale avec mémorisation
dico = {}


def binomial(n, p):
    """Calcule le coefficient binomial"""
    if p == 0 or p == n:
        return 1
    elif (n, p) in dico:
        return dico[(n, p)]
    else:
        dico[(n, p)] = binomial(n - 1, p) + binomial(n - 1, p - 1)
        return dico[(n, p)]


def bernstein(x, n, k):
    return binomial(n, k) * x**k * (1 - x)**(n - k)


def Bezier0(Ps, t):
    """ Renvoie le point de la courbe de Bézier d'ordre quelconque
    t varie entre 0 et 1"""
    x, y = 0, 0
    n = len(Ps) - 1
    for k in range(n + 1):
        xk, yk = Ps[k]
        b = bernstein(t, n, k)
        x += b * xk
        y += b * yk
    return (x, y)


def coord(Ps):
    """Renvoie suivant le pas régulier choisi les coordonées succéssives de la courbe de Bézier"""
    Ps = [(float(x), float(y)) for x, y in Ps]
    return [Bezier0(Ps, t) for t in np.arange(0, 1.1, 0.1)]


# Fonction pour afficher la courbe de Bézier
def plot_bezier(Ps, show_points=True):
    """Affiche la courbe de Bézier"""
    ts = np.linspace(0, 1, 500)
    Qs = [Bezier0(Ps, t) for t in ts]
    xs, ys = zip(*Qs)
    plt.plot(xs, ys, 'k', label="Trajectoire Bézier")
    if show_points:
        for (x, y) in Ps:
            plt.scatter(x, y, color="red")
    return xs,ys
```

```
084|
085| ## Points de contrôle pour la trajectoire de Bezier
086| # #Points fixes
087| # Pc1 = (-10, 3)
088| # Pc2 = (-17, 2.5)
089| # Pc3 = (-15, 2)
090| # Pc4 = (15, 3.5)
091| # Pc6 = (-10, -1)
092| # Pc7 = (15, 1)
093|
094| #Points aléatoires
095| Pcx1 =np.random.uniform(-15, -5)
096| Pcy1 = np.random.uniform(2.75, 3.25)
097| Pcx2 = np.random.uniform(-22, -12)
098| Pcy2 = np.random.uniform(2.25, 2.75)
099| Pcx3 = np.random.uniform(-20, -10)
100| Pc1 = (Pcx1, Pcy1)
101| Pc2 = (Pcx2, Pcy2)
102| Pc3 = (Pcx3, 2)
103|
104|
105|
106| # Choix des points de contrôle en fonction de la
position de l'avion
107| if avion_y > 2:
108|     Ps = [Avion, Pc1,Pc2,Pc3,Radar]
109| else:
110|     Ps = [Avion, Pc3, Pc4, Pc6, Pc7, Radar]
111|
112|
113| #Affichage de la courbe de Bézier uniquement si
l'avion est en approche
114| # if abs(avion_y - 2) > 0.01:
115| #     plot_bezier(Ps)
116| # else:
117| #     print("Atterissage terminé")
118|
119|
120|
121| ## Fonctions auxiliaires à la fonction coût
122| def extractionx(Ps)->list:
123|     """Extrait toutes les coordonées suivant x d'une
liste de points"""
124|     x_list=[]
125|     for i in range(0,len(Ps)):
```



```
126|         x_list.append(Ps[i][0])
127|     return x_list
128|
129|
130| def extractiony(Ps)->list:
131|     """Extrait toutes les coordonées suivant y d'une
liste de points"""
132|     y_list=[]
133|     for i in range(0,len(Ps)):
134|         y_list.append(Ps[i][1])
135|     return y_list
136|
137|
138| Ps1=np.array(Ps)
139| x_t = np.gradient(Ps1[:,0]) #dérivée 1ere
140| y_t = np.gradient(Ps1[:,1])
141|
142|
143| def velo(Ps1):
144|     """renvoie une liste de vecteurs vitesse"""
145|     return np.array([[x_t[i], y_t[i]] for i in
range(len(x_t))])
146|
147|
148| def speed(Ps1):
149|     """Utilise la norme euclidienne pour renvoyer un
tableau de vitesse scalaires en chaque point de la
trajectoire"""
150|     return np.sqrt(x_t**2+y_t**2)
151|
152|
153| def tangent(Ps1):
154|     """Retourne une liste de vecteurs tangents
unitaires le long de la trajectoire définie par les points
de Ps1"""
155|     speeds=speed(Ps1)
156|     return np.array([[x_t[i] / speeds[i], y_t[i] /
speeds[i]] for i in range(len(speeds))])
157|
158|
159| def courbure(trajectoire):
160|     """Retourne la courbure associée à une trajectoire
"""
161|     Ps1 = np.array(trajectoire)  # Conversion en numpy
array
```

```
    x_t = np.gradient(Ps1[:, 0])  # Dérivée première en x
    y_t = np.gradient(Ps1[:, 1])  # Dérivée première en y
    xx_t = np.gradient(x_t)  # Dérivée seconde en x
    yy_t = np.gradient(y_t)  # Dérivée seconde en y
    courbure_vals = np.abs(xx_t * y_t - x_t * yy_t) / ((x_t**2 + y_t**2) ** 1.5)
    return courbure_vals


## Fonction coût
def dist(trajectoire) -> float:
    """Renvoie la distance parcourue par l'avion lours de son atterrissage"""
    X = extractionx(trajectoire)
    Y = extractiony(trajectoire)
    S = sum((X[k] - X[k - 1])**2 + (Y[k] - Y[k - 1])**2 for k in range(1, len(trajectoire)))
    return np.sqrt(S)


def variationcourbure(trajectoire) -> float:
    """ Mesure à quel point la courbure change brutalement le long de la trajectoire"""
    courbures = courbure(trajectoire)  # Calcul des courbures sur la trajectoire
    return sum(abs(courbures[i] - courbures[i - 1]) for i in range(1, len(courbures))) / len(courbures)


def cout(Ps):
    """Renvoie le coût associé à une trajectoire à partir des critères physiques"""
    trajectoire = coord(Ps)

    # Critères physiques
    max_courbure = max(courbure(trajectoire)) / 100
    rayon_min = 1 / max_courbure if max_courbure != 0 else float('inf')
    distance = dist(trajectoire) / 10
    variation = variationcourbure(trajectoire) / 10

    # Contrainte de courbure minimale
    if rayon_min < 4:  # Rayon de courbure minimal
```



```
imposé
        return 1e6

    # Contrainte de chronologie sur les abscisses (éviter que la courbe "recule")
    x0, x1, x2 = Ps[0][0], Ps[1][0], Ps[2][0]
    if x1 > x0 or x2 > x1:
        return 1e6

    # Contrainte de descente progressive en ordonnée (y)
    y0, y1, y2, y3 = Ps[0][1], Ps[1][1], Ps[2][1], Ps[3][1]
    if not (y0 > y1 > y2 > y3):
        return 1e6

    # Pondération des critères
    d = 0.01  # poids de la courbure max
    e = 1.0   # poids de la distance
    f = 1.0   # poids de la variation de courbure

    return d * max_courbure + e * distance + f * variation


##Fonctions auxiliaires à la génération d'atterrissage idéal
def indicemin(L)->int:
    """Trouve l'indice associé au minimum d'une liste"""
    imin=0
    for k in range(len(L)):
        if L[k]<L[imin]:
            imin=k
    return imin


def indicemax(L)->int:
    """Trouve l'indice associé au maximum d'une liste"""
    imax=0
    for k in range(len(L)):
        if L[k]>L[imin]:
            imax=k
    return imax
```

```
234|
235| def tri_fusion(pop) -> list:
236|     """Algorithmes de tri fusion et du fusion adaptés
à la comparaison de coûts entre 2 atterrissages"""
237|     pop=population(Avion,Radar, 10)
238|     if len(pop) <= 1:  # Cas de base
239|         return pop
240|     else:
241|         # Division
242|         i_milieu = len(pop) // 2
243|         L1 = tri_fusion(pop[:i_milieu])
244|         L2 = tri_fusion(pop[i_milieu:])
245|         if not L1 or not L2:
246|             return L1 + L2
247|
248|         return fusion(L1, L2)
249|
250| def fusion(LT1: list, LT2: list) -> list:
251|     LT = []
252|     i, j = 0, 0
253|
254|     while i < len(LT1) and j < len(LT2):
255|         if cout(LT1[i]) <= cout(LT2[j]):
256|             LT.append(LT1[i])
257|             i += 1
258|         else:
259|             LT.append(LT2[j])
260|             j += 1
261|
262|     # Ajout les éléments restants
263|     LT.extend(LT1[i:])
264|     LT.extend(LT2[j:])
265|     return LT
266|
267| def generation_trajectoire(Avion,Radar):
268|     """Permet de générer une trajectoire par
génération aléatoire des points de contrôle"""
269|     Pcx1 = np.random.uniform(-15, -5)
270|     Pcy1 = np.random.uniform(2.75, 3.25)
271|     Pcx2 = np.random.uniform(-22, -12)
272|     Pcy2 = np.random.uniform(2.25, 2.75)
273|     Pcx3 = np.random.uniform(-20, -10)
274|     Pc1 = (Pcx1, Pcy1)
275|     Pc2 = (Pcx2, Pcy2)
276|     Pc3 = (Pcx3, 2)
```



```
277|     trajectoire=[Avion, Pc1,Pc2,Pc3,Radar]
278|     return trajectoire
279|
280| def generation_population(Avion, Radar, n):
281|     """" Crée une population de trajectoires triées
par coût croissant
282|     n= nombre d'individus que à générer"""
283|     popu = []
284|     costs = []
285|     population=popu,costs
286|
287|     for _ in range(n):
288|
trajectoire=generation_trajectoire(Avion,Radar)
289|         popu.append(trajectoire)
290|         costs.append(cout(trajectoire))
291|     pop_triee, costs_tries = zip(*sorted(zip(popu,
costs), key=lambda x: x[1]))
292|     return pop_triee,costs_tries
293|
294| def crossover(ind1, ind2) -> list:
295|     """
296|     Combine deux individus (trajectoires) pour en
générer un enfant.
297|     Chaque point de contrôle est une moyenne pondérée
des deux parents.
298|     On force l'abscisse du point P3 à rester
strictement inférieure à -2.
299|
300|     Retourne une nouvelle trajectoire enfant.
301|     """
302|     alpha = random.random()
303|     enfant = [(alpha * k1[0] + (1 - alpha) * k2[0],
304|                alpha * k1[1] + (1 - alpha) * k2[1])
for k1, k2 in zip(ind1, ind2)]
305|
306|     # Forcer P3 à être suffisamment avant la piste
307|     x_p3 = enfant[3][0]
308|     if x_p3 >= -2:
309|         enfant[3] = (-2 - abs(x_p3 + 0.1), enfant[3]
[1])
310|
311|     return enfant
312|
313|
```

```
314| def mutation(Ps, p_mut=0.2, sigma=0.5):
315|     """
316|     Applique une mutation gaussienne sur les points de
contrôle d'une trajectoire.
317|     Le point P3 est contraint à avoir une abscisse
strictement inférieure à -2.
318|
319|     - p_mut : probabilité de mutation
320|     - sigma : intensité de la mutation (écart-type)
321|
322|     Retourne une version mutée (ou non) de la
trajectoire.
323|     """
324|     if np.random.rand() < p_mut:
325|         PsmX1 = Ps[1][0] + np.random.normal(0, sigma)
326|         PsmY1 = Ps[1][1] + np.random.normal(0, sigma)
327|         PsmX2 = Ps[2][0] + np.random.normal(0, sigma)
328|         PsmY2 = Ps[2][1] + np.random.normal(0, sigma)
329|         PsmX3 = Ps[3][0] + np.random.normal(0, sigma)
330|
331|         # Forcer P3 à être situé strictement avant x =
-2
332|         if PsmX3 >= -2:
333|             PsmX3 = -2 - abs(PsmX3 + 0.1)
334|
335|         Psm1 = (PsmX1, PsmY1)
336|         Psm2 = (PsmX2, PsmY2)
337|         Psm3 = (PsmX3, Ps[3][1])  # on garde y de P3
inchangé
338|
339|         return [Ps[0], Psm1, Psm2, Psm3, Ps[-1]]
340|     else:
341|         return Ps
342|
343|
344|
345|
346| ## Approches permettant de déterminer les points de
contrôle idéaux
347| def attideal(n):
348|     """ Génére de façon aléatoire des points de
contrôle
349|     n= nombre de simulations
350|     cost= liste contenant les coûts successifs
351|     Datas= liste contenant les points de contrôle
```



```
associés
352|     Retourne l'atterrissage possédant le coût le plus
faible
353|     """
354|     cost = []
355|     Datas = []
356|     for _ in range(n):
357|         Pcx1 =np.random.uniform(-15, -5)
358|         Pcy1 = np.random.uniform(2.75, 3.25)
359|         Pcx2 = np.random.uniform(-22, -12)
360|         Pcy2 = np.random.uniform(2.25, 2.75)
361|         Pcx3 = np.random.uniform(-20, -10)
362|         Pc1 = (Pcx1, Pcy1)
363|         Pc2 = (Pcx2, Pcy2)
364|         Pc3 = (Pcx3, 2)
365|         # Construction d'une nouvelle trajectoire avec
ces points de contrôle
366|         Ps = [Avion, Pc1, Pc2, Pc3, Radar]
367|         coord = [Bezier0(Ps, t) for t in np.arange(0,
1.1, 0.1)]  # Calcul de la courbe
368|         # Calcul du coût pour cette simulation
369|         cost.append(cout(Ps))
370|         Datas.append(Ps)
371|     # Trouver la configuration avec le coût minimal
372|     winner_index = indicemin(cost)
373|     return Ps
374|
375|
376| def attidealv2(n):
377|     """Modifie la trajectoire idéale si son coût est
inférieur à celle de sa prédecesseure
378|     n= nombre de simulations
379|     epsilon= intervalle selon lequel on fait varier
les points
380|     Retourne l'atterrissage possédant le coût le plus
faible
381|         """
382|     epsilon=0.1
383|     Pcx1 =np.random.uniform(-15, -5)
384|     Pcy1 = np.random.uniform(2.75, 3.25)
385|     Pcx2 = np.random.uniform(-22, -12)
386|     Pcy2 = np.random.uniform(2.25, 2.75)
387|     Pcx3 = np.random.uniform(-20, -10)
388|     Pc1 = (Pcx1, Pcy1)
389|     Pc2 = (Pcx2, Pcy2)
```

```
390|     Pc3 = (Pcx3, 2)
391|     Ps1 = [Avion, Pc1, Pc2, Pc3, Radar]
392|     coordonnes = coord(Ps1)
393|     for i in range(n):
394|         Pcx12 =np.random.uniform(Pcx1-epsilon,
Pcx1+epsilon)
395|         Pcy12 = np.random.uniform(Pcy1-epsilon,
Pcy1+epsilon)
396|         Pcx22 = np.random.uniform(Pcx2-epsilon,
Pcy1+epsilon)
397|         Pcy22 = np.random.uniform(Pcy2-epsilon,
Pcy2+epsilon)
398|         Pcx32 = np.random.uniform(Pcx3-epsilon,
Pcx3+epsilon)
399|         Pc12 = (Pcx12, Pcy12)
400|         Pc22 = (Pcx22, Pcy22)
401|         Pc32 = (Pcx32, 2)
402|         Ps2 = [Avion, Pc12, Pc22, Pc32, Radar]
403|         if cout(Ps2)<cout(Ps1):
404|             Ps1=Ps2
405|     return Ps1
406|
407|
408| def AG(Avion, Radar, n, population_size=50, p_mut=0.2,
sigma=1.0):
409|     """
410|     Algorithme génétique pour optimiser les
trajectoires d'atterrissage.
411|
412|     - Avion, Radar : points fixes
413|     - n : nombre de générations
414|     - population_size : taille de la population
415|     - p_mut : probabilité de mutation
416|     - sigma : intensité des mutations
417|
418|     Retourne une liste des points de contrôle et des
coûts au fil des générations.
419|     """
420|
Individus=generation_population(Avion,Radar,population_size
)[0]
421|     for _ in range(n):
422|         Population=list(Individus[:4])
423|         enfants= []
424|         for i in range(len(Population)):
```



```
425|             for j in range(i+1,len(Population)):
426|
m=crossover(Population[i],Population[j])
427|
enfantmuté=mutation(m,p_mut=p_mut,sigma=sigma)
428|                 enfants.append(enfantmuté)
429|         Population.extend(enfants)
430|         couts=[cout(h) for h in Population]
431|         population_triee, costs_tries =
zip(*sorted(zip(Population, couts), key=lambda x: x[1]))
432|         Individus =
list(population_triee[:population_size])
433|         couts.append(costs_tries[0])
434|
435|     return Individus,couts
436|
437|
438|
439|
440| ## On stocke les coûts successifs
441| def attidealc(n):
442|     """ Génére de façon aléatoire des points de
contrôle
443|     n= nombre de simulations
444|     cost= liste contenant les coûts successifs
445|     Datas= liste contenant les points de contrôle
associés
446|     Retourne une liste où sont stockés les coûts au
fil des générations
447|     """
448|     cost = []
449|     Datas = []
450|     for _ in range(n):  # n simulations
451|         Pcx1 =np.random.uniform(-15, -5)
452|         Pcy1 = np.random.uniform(2.75, 3.25)
453|         Pcx2 = np.random.uniform(-22, -12)
454|         Pcy2 = np.random.uniform(2.25, 2.75)
455|         Pcx3 = np.random.uniform(-20, -10)
456|         Pc1 = (Pcx1, Pcy1)
457|         Pc2 = (Pcx2, Pcy2)
458|         Pc3 = (Pcx3, 2)
459|         # Construire une nouvelle trajectoire avec ces
points de contrôle
460|         Ps = [Avion, Pc1, Pc2, Pc3, Radar]
461|         coord = [Bezier0(Ps, t) for t in np.arange(0,
```

```
1.1, 0.1)]  # Calcul de la courbe
462|          # Calcul du coût pour cette simulation
463|          cost.append(cout(Ps))
464|          Datas.append(Ps)
465|      return cost
466|
467|  def attidealv2c(n):
468|      """Modifie la trajectoire idéale si son coût est
inférieur à celle de sa prédecesseure
469|      n= nombre de simulations
470|      epsilon= intervalle selon lequel on fait varier
les points
471|      Retourne une liste où sont stockés les coûts au
fil des générations
472|          """
473|
474|      epsilon = 0.1
475|      Pcx1 = np.random.uniform(-15, -5)
476|      Pcy1 = np.random.uniform(2.75, 3.25)
477|      Pcx2 = np.random.uniform(-22, -12)
478|      Pcy2 = np.random.uniform(2.25, 2.75)
479|      Pcx3 = np.random.uniform(-20, -10)
480|
481|      Pc1 = (Pcx1, Pcy1)
482|      Pc2 = (Pcx2, Pcy2)
483|      Pc3 = (Pcx3, 2)
484|      Ps1 = [Avion, Pc1, Pc2, Pc3, Radar]
485|
486|      couts = [cout(Ps1)]  # On commence avec le coût
initial
487|
488|      for _ in range(n - 1):  # Générer n-1 nouvelles
valeurs
489|          Pcx12 = np.random.uniform(Pcx1 - epsilon, Pcx1
+ epsilon)
490|          Pcy12 = np.random.uniform(Pcy1 - epsilon, Pcy1
+ epsilon)
491|          Pcx22 = np.random.uniform(Pcx2 - epsilon, Pcy1
+ epsilon)
492|          Pcy22 = np.random.uniform(Pcy2 - epsilon, Pcy2
+ epsilon)
493|          Pcx32 = np.random.uniform(Pcx3 - epsilon, Pcx3
+ epsilon)
494|
495|          Pc12 = (Pcx12, Pcy12)
```



```
496|          Pc22 = (Pcx22, Pcy22)
497|          Pc32 = (Pcx32, 2)
498|          Ps2 = [Avion, Pc12, Pc22, Pc32, Radar]
499|
500|          if cout(Ps2) < cout(Ps1):
501|              Ps1 = Ps2
502|
503|          couts.append(cout(Ps1))  # Toujours ajouter
une valeur pour arriver à 100
504|      return couts
505|
506|  def AGc(Avion, Radar, n, population_size=10,
p_mut=0.2, sigma=1.0):
507|      """
508|      Algorithme génétique pour optimiser les
trajectoires d'atterrissage.
509|
510|      - Avion, Radar : points fixes
511|      - n : nombre de générations
512|      - population_size : taille de la population
513|      - p_mut : probabilité de mutation
514|      - sigma : intensité des mutations
515|
516|      Retourne une liste des coûts au fil des
générations.
517|      """
518|      # Génération initiale
519|      Individus = generation_population(Avion, Radar,
population_size)[0]
520|      couts = []
521|
522|      for _ in range(n):
523|          # Sélection des meilleurs individus (ex: les 4
meilleurs)
524|          Population = list(Individus[:4])
525|          enfants = []
526|
527|          # Croisement entre individus
528|          for i in range(len(Population)):
529|              for j in range(i+1, len(Population)):
530|                  enfant = crossover(Population[i],
Population[j])
531|
532|                  # Appliquer la mutation avec proba
p_mut
```

```
533|                 enfant_muté = mutation(enfant,
p_mut=p_mut, sigma=sigma)
534|                 enfants.append(enfant_muté)
535|
536|         # Mise à jour de la population
537|         Population.extend(enfants)
538|
539|         # Évaluation des coûts
540|         couts_population = [cout(h) for h in
Population]
541|
542|         # Sélection des meilleurs individus pour la
prochaine génération
543|         population_triee, costs_tries =
zip(*sorted(zip(Population, couts_population), key=lambda
x: x[1]))
544|
545|         # On garde les `population_size` meilleurs
546|         Individus =
list(population_triee[:population_size])
547|
548|         # Stocker le meilleur coût pour suivre
l'évolution
549|         couts.append(costs_tries[0])
550|
551|     return couts
552|
553| ## Graphe comparatif des différentes méthodes
554| nbr_de_simulations = 2000 # Nombre de simulations
555|
556| # Récupérer les coûts avec 100 valeurs
557| cout_attideal = attidealc(nbr_de_simulations)
558| cout_attidealv2 = attidealv2c(nbr_de_simulations)
559| cout_AG = AGc(Avion, Radar, nbr_de_simulations)
560|
561| # Vérification et ajustement des longueurs
562| min_length = min(len(cout_attideal),
len(cout_attidealv2), len(cout_AG))
563| cout_attideal = cout_attideal[:min_length]
564| cout_attidealv2 = cout_attidealv2[:min_length]
565| cout_AG = cout_AG[:min_length]
566|
567| # Axe des itérations
568| iterations = range(min_length)
569|
```



```
570| # Tracé des courbes
571| plt.plot(iterations, cout_attideal, color="red",
label="Génération aléatoire")
572| plt.plot(iterations, cout_attidealv2, color="purple",
label="Génération intelligente")
573| plt.plot(iterations, cout_AG, color="green",
label="Algorithme génétique")
574| plt.yscale('log') #Echelle logarithmique
575|
576| # Configuration du graphique
577| plt.xlabel("Nombre d'itérations")
578| plt.ylabel("Coût")
579| plt.title("Evolution du coût en fonction du nombre
d'itérations ")
580| plt.legend()
581| plt.grid(True)
582| plt.show()
583|
584|
585| ## Affichage de l'atterissage souhaité sur un graphe
586|
587| def affichage(CP):
588|     """Prend en argument une liste de points de
contrôle et renvoie l'atterrisage correspondant
589|     CP peut être de la forme:
590|     CP=AG(Avion,Radar,3)[0][0]
591|     CP=attideal(3)
592|     CP=attidealv2(3)
593|     CP=Ps"""
594|     #Avion
595|     Avion=(avion_x,avion_y)
596|     # Piste
597|     X = np.linspace(0, 5, 500)
598|     piste_y = [2] * 500
599|     # Affichage de la piste
600|     plt.plot(X, piste_y, color="green", label="Piste
d'atterrissage")
601|     # #Affichage du radar en violet
602|     # plt.scatter(radar_x, radar_y, color="purple",
label='Radar')
603|
604|     # Affichage de l'avion et des points de contrôle
605|     plt.scatter(avion_x, avion_y, color="red",
label='Points de contrôle')
606|
```

```
607|     #Courbe de Bézier
608|     plot_bezier(CP)
609|
610|     # Finalisation de l'affichage
611|     plt.xlabel('Coordonnée x')
612|     plt.ylabel('Coordonnée y')
613|     plt.title("Piste d'un aéroport vue du dessus")
614|     plt.legend()
615|     plt.grid(True)
616|     plt.show()
```


42